Scout Shopping Assistant

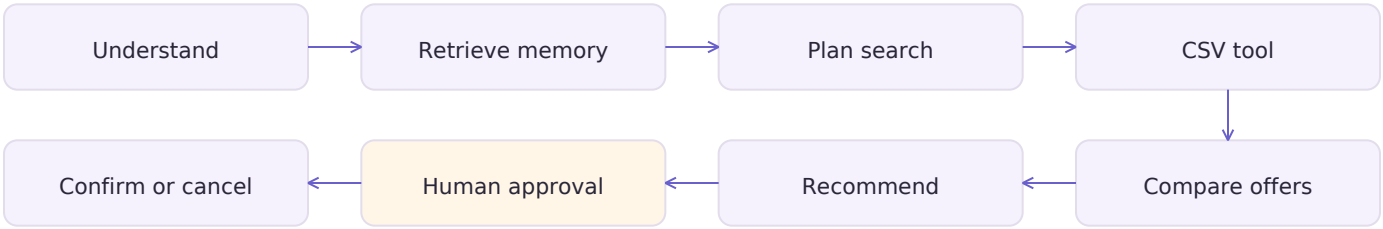
ITS 58100 | Project 1 | One memory enabled agent

Scout helps a user choose wireless headphones by turning a natural-language request into a constrained search and a justified comparison. One Gemini-powered LangGraph workflow uses a CSV information source and approved preferences across separate interactions. The local webapp adds Shopping, Catalog, Memory, and Architecture tabs.

Problem and input

Example: “Find wireless headphones under $150 for studying; prioritize noise cancellation.” The sample catalog contains 110 fictional offers across 36 models and three simulated retailers. Output includes a shortlist, known item-and-shipping totals before tax, exclusions, uncertainty, and a proposed simulated purchase.

Workflow and execution control



Separate memory path: Form preference > Human approval > Save or finish without a write

Conditional paths: clarify missing budget; stop unsupported requests or unmatched offers.

Full graph: Architecture tab, architecture.svg, and the generated architecture.mmd file.

The implementation retains the labs’ TypedDict state, small node functions, partial updates, conditional edges, and audit log. Gemini extracts requirements and writes search steps, recommendations, and memory proposals. Python binds the exact validated filters to the plan and enforces schemas, decimal arithmetic, and approved offer IDs. The executable plan drives the CSV tool and comparison policy.

The graph pauses with interrupt() before a simulated purchase or memory write. A strict boolean passed through Command(resume=...) selects approval or rejection in the same thread. The side effect runs in a later node and checks approval again. Rejection cancels; no real order, payment, or retailer action exists.

Design decisions

Current explicit constraints override remembered defaults. Unknown shipping cannot win a delivered-cost comparison. Only equivalent model, variant, and condition offers are interchangeable. Hard constraints precede ranking, and no-match requests stop without relaxing the budget. Gemini errors stay visible and never silently enable the explicitly labeled sample engine.

Memory and verification

Thread state and reusable preferences

InMemorySaver checkpoints each thread_id: request, constraints, plan, offers, approval, status, and events. InMemoryStore holds approved records under the user’s shopping namespace. A new thread retrieves relevant active records without inheriting the old approval or execution state. Both backends clear on process restart.

Memory formation separates STORE candidates, TEMPORARY context, and HUMAN_REVIEW. Only supported, canonical fit, condition, feature, and ranking preferences can be saved after approval. Ranking uses relevance, importance, and deterministic timestamp-based recency. A correction supersedes the old active record while preserving its source, approval time, and replacement link.

Recorded scenarios

Scenario	Expected behavior	Observed sample result
Normal request	Pause, then confirm simulation	SIMULATED_CONFIRMED
Later thread with two records	Apply over-ear and new-only preferences	2 records; O072 selected
Rejected purchase	No confirmation node	CANCELLED
Approved correction	Replace new-only with any condition	any
Temporary exception	No reusable write	NOT_STORED
Rejected memory	Keep store unchanged	MEMORY_REJECTED
No suitable offer	Stop without action	NO_MATCHES

24 local regression checks passed using the labeled sample engine and invalid-output stubs. They cover real graph pauses, strict consent, repeat-resume prevention, totals, unknown shipping, memory correction and history, temporary exceptions, duplicate handling, and scoped access. The executed notebook has seven code cells. Demo decisions are scripted mock-human inputs through actual interrupts.

Earlier live Gemini scenarios passed on the original six-model catalog: normal confirmation, two approved records, retrieval, rejection, correction, and no matches. Preserved evidence/gemini identifies those original live states; the expanded catalog was verified with the sample engine. Model: gemini-3.1-flash-lite. Captured reruns may vary; earlier tests encountered provider quota and high-demand responses.

Limitations and contributions

The app is a local educational prototype. Simulated user IDs and ownership checks are not production authentication. Memory is not durable, catalog data is fictional, prices exclude tax, and Gemini prose may require review. No multiple-agent coordination, shared memory, real shopping API, or checkout is included.

The student supplied the shopping domain, CSV scope, web frontend, and architecture requirements. Implementation and documentation were prepared with Codex assistance. Confirm actual team names and individual contributions before submission.